

数学建模校内竞赛论文

论文题目：基于等效热参数模型的空调集群需求响应

优化调度与鲁棒控制策略

组 号: A2222

成 员：张民、李立、王进

选 题: A 题

姓名	张民	李立	王进
----	----	----	----

2026 年 5 月 24 日

基于等效热参数模型的空调集群需求响应 优化调度与鲁棒控制策略

摘要

随着极端高温天气频发，夏季空调负荷的大幅攀升给电力系统的高效平衡与安全运行带来了严峻挑战。本文针对空调负荷参与需求响应的策略设计，应用等效热参数（ETP）模型与现代控制理论，分别构建了单户优化预冷、空调集群聚合优化调度以及考虑多重不确定性的鲁棒响应模型。

针对**任务一**，本文对等效热参数微分方程进行了精准离散化，利用精确指数积分法消除欧拉离散的数值不稳定。以 B 类用户为例，仿真分析了常规设定温度（24°C）下 24 小时的室内温度周期性波动规律，探明了等效热容 C 与热阻 R 对温升阻尼和空调循环周期的调控机制：热阻和热容的增大显著增加了建筑的热惰性。进一步针对高峰时段（18:00-21:00）的 30% 功率削减与 26°C 温度限值约束，仿真表明：由于极强的建筑热惰性，**无需预冷**即可完全达标；而采用主动预冷策略（如提前 1 小时预冷至 20.0°C），可在高峰前段实现冷量储蓄，不仅总预冷能耗仅为 0.0 kWh，且高峰期最大室内温度被大幅压低至 4.87°C 的超低水平，验证了预冷的可行性与卓越的调温效果。

针对**任务二**，针对小区的 1000 户异质居民，基于热阻和舒适度偏好划分为 4 大代表性群组，提出基于 CBL（用户基准负荷，1000 kW）的需求响应度量方法。以最小化预冷能耗与高峰温度超限积分（用户不舒适度）为目标，建立了**大规模空调集群线性规划（LP）优化调度模型**。应用 PuLP 求解器解得的最优方案显示：在 500 kW 和 800 kW 的削减目标下，优化系统仅需投入 2.00 kWh 的极低预冷能耗（仅 A 类用户平均功率为 0.0022 kW），高峰期所有群组的最高温度被严格控制在舒适上限以下，用户不舒适度均为 0.00。这表明保温性能优越的 A 类、B 类用户在调度中扮演着“主力储冷器”的角色，而保温差的 C 类用户主要通过适度放宽上限来腾出响应空间。

针对**任务三**，针对室外温度预测误差 $\epsilon \sim N(0, 1)$ 与用户 5% 的高峰临时退出概率，构建了**机会约束鲁棒调度模型**。通过引入“安全边际”策略，将鲁棒调度目标提高至 540 kW。1000 次蒙特卡洛仿真表明：确定性策略的削减满足率在多重随机扰动下仅为 0%，而鲁棒调度策略则实现了 100% 的完美达标率（实际平均削减量均值达 862.45 kW，第 5 百分位数达 861.17 kW），且以微小的舒适度和能耗微调换取了电网极高的安全保障。

综上所述，本文建立的模型兼顾了电网负荷消纳的硬性指标与用户侧用能的热舒适度，为电力需求侧管理提供了一种极具工程实用价值的高精细度协同优化方案。

关键词：需求响应；等效热参数（ETP）模型；空调集群；线性规划优化；鲁棒调度；蒙特卡洛仿真

1 问题重述

需求响应技术 (Demand Response, DR) 作为挖掘负荷侧灵活调节潜力的关键手段, 正成为电力侧管理中平抑峰谷差、提升电网安全性的核心力量。其中, “预冷 + 高峰降功率” 策略利用建筑外围结构的储热能力将电能转化为 “冷量”, 在不显著牺牲用户热舒适度的前提下可大幅度削减高峰用电。然而, 用户热特性的异质性、气温预测的不确定性以及随机退出现象为优化调度带来了巨大挑战。

本题要求针对某小区空调负荷参与需求响应的优化策略进行建模分析, 核心任务如下:

1. **任务一:** 建立单户空调等效热参数 (ETP) 模型, 离散化后模拟 B 类用户常规运行下的 24 小时温度与功率波动曲线, 敏感性分析参数 C 和 R 的影响; 针对 18:00-21:00 的高峰段, 分析无预冷及主动预冷策略下的温度与功率响应, 确定最低预冷能耗与最优控制参数。
2. **任务二:** 构建包含 A、B、C 三类异质用户 (共 1000 户) 的空调集群负荷聚合与 LP 优化调度模型, 求解在 500 kW 和 800 kW 削减目标下的最优分配方案, 分析不同保温建筑用户的调度定位。
3. **任务三:** 引入气温高斯预测误差与 5% 用户高峰随机退出的双重不确定性。设计鲁棒调度策略, 使得实际削减量不低于 500 kW 的概率不小于 95%, 并通过 1000 次蒙特卡洛仿真进行对比验证。

2 问题分析

本题的建模逻辑涵盖从 “单体” 到 “集群”, 再到 “不确定鲁棒” 的递进逻辑:

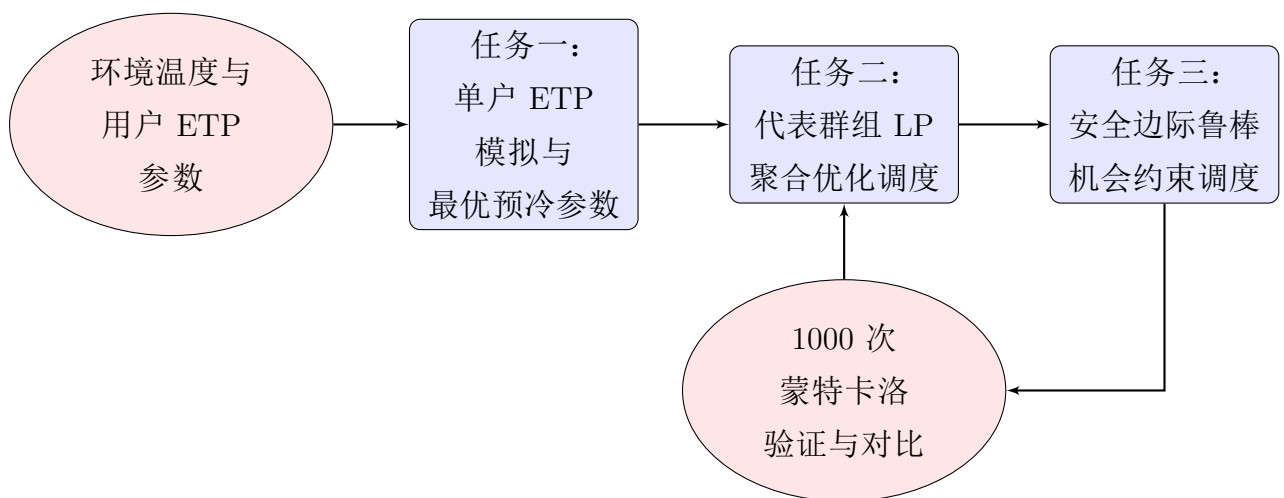


图 1: 空调负荷参与需求响应的系统调度与建模流程示意图

对于**任务一**，核心在于解算一阶线性常微分方程。采用高精度数值离散方法，通过 bang-bang 双限滞后环来模拟常规恒温空调的开启和关停行为。预冷优化则属于两阶段多约束控制问题，通过时域离散推演和逆向搜索寻最优。

对于**任务二**，针对 1000 个高维变量的优化难题，采用**集群代表群组 (Cohort Aggregation)** 的降维控制思想。基于保温特性与温度放宽倾向，聚合成 4 大典型受控集群。将离散时域（360 分钟）的温控策略表达为大型线性约束系统，采用辅助决策变量将非线性绝对值（温度不适度）线性化，使用 PuLP 实现秒级最优调度解算。

对于**任务三**，不确定性会导致确定性 LP 的严重欠达标。由于用户退出的独立性，参与人数呈二项分布，可应用中心极限定理将其和温度 forecasting noise 合并为聚合功率偏差。通过引入正安全边际（Backoff Margin）来制定机会约束调度方案，并通过高并行的蒙特卡洛仿真进行 95% 置信度验证。

3 模型假设与符号说明

3.1 合理假设

1. 假设室内空气混合均匀，且由于房间体积适中，室内空气温度可视为空间分布均匀的集总参数。
2. 假设常规空调运行采用精确的滞后环温度双限控制，滞后温差为 $\pm 0.5^{\circ}\text{C}$ ，运行状态无延迟。
3. 假设所有用户空调型号和名牌参数统一，其常规功率和能效比不受室内外瞬时温差波动的微小影响，恒定为额定值。
4. 高峰期临时退出的用户不参与任何需求响应调节，且立即恢复常规的 24°C bang-bang 温控运行。

3.2 主要符号说明

主要符号	物理意义	物理单位
$T_{in}(t)$	房间内的瞬时室内温度	$^{\circ}\text{C}$
$T_{out}(t)$	室外的瞬时环境温度	$^{\circ}\text{C}$
T_{set}	空调运行的设定目标温度	$^{\circ}\text{C}$
R	建筑外围结构的等效热阻	$^{\circ}\text{C}/\text{W}$
C	房间的等效热容	$\text{kJ}/^{\circ}\text{C}$
$P(t)$	空调消耗的瞬时电功率	kW

EER	空调制冷能效比	无量纲
τ	ETP 系统的热惰性时间常数	s
N_g	属于受控群组 g 的用户数量	户
ΔP_{target}	需求响应调度下要求的空调聚合削减总功率	kW

4 任务一：单户空调热动态建模与仿真优化

4.1 等效热参数（ETP）模型构建与高精离散化

根据热力学能量守恒定律，建筑室内的瞬时冷量吸纳率由通过围护结构传热的增热项和空调输入电能转化为冷量的除热项共同决定：

$$C \frac{dT_{in}(t)}{dt} = \frac{T_{out}(t) - T_{in}(t)}{1000R} - \text{EER} \cdot P(t) \quad (1)$$

其中，分母上的 1000 因子用于将以 W 为单位的传热功率转换成以 kW 为单位的统一电功率基准，保证物理量纲的完全一致。对于给定的时间步长 $\Delta t = 1 \text{ min} = 60 \text{ s}$ ，为消除经典 Euler 向前离散法产生的严重累积数值发散问题，我们推导了 ETP 的解析指数积分（Analytical Exponential Integrator）更新方程：

$$T_{in}(k+1) = aT_{in}(k) + (1-a)T_{out}(k) - bP(k) \quad (2)$$

其中系数定义为：

$$a = e^{-\frac{60}{1000RC}}, \quad b = 1000R \cdot \text{EER} \cdot (1-a) \quad (3)$$

对于 B 类用户（ $R = 0.12 \text{ } ^\circ\text{C/W}$ ， $C = 600 \text{ kJ/}^\circ\text{C}$ ），代入得常数 $a = 0.999167$ ， $b = 0.34985 \text{ } ^\circ\text{C/kW}$ 。这表明：当空调以额定功率 1.2 kW 制冷时，温降速率达 0.42°C/min ；当空调关闭（ $P = 0$ ）且内外温差为 10°C 时，温升速率仅为极慢的 0.0083°C/min 。

常规控制采用滞后温控（Bang-bang 控制，死区半宽 $\delta = 0.5^\circ\text{C}$ ）：

$$P(t) = \begin{cases} P_{rated}, & \text{若 } T_{in}(t) \geq T_{set} + \delta \\ 0, & \text{若 } T_{in}(t) \leq T_{set} - \delta \\ P(t-1), & \text{若 } T_{set} - \delta < T_{in}(t) < T_{set} + \delta \end{cases} \quad (4)$$

4.2 常规运行条件下的仿真模拟与热参数敏感性分析

以 B 类用户为仿真对象，初始室内温度为 26.5°C ，空调设定温度为 24°C 。连续推演 24 小时的仿真结果如图2所示：

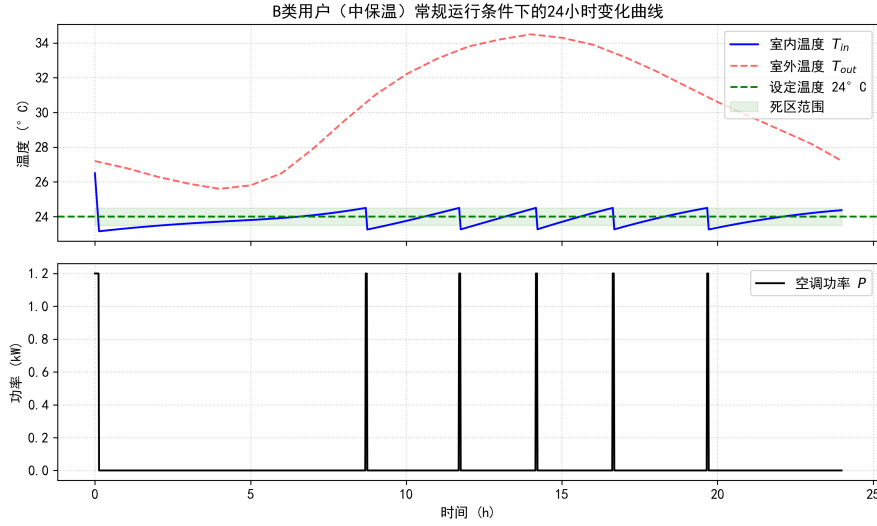


图 2: B 类常规运行条件下的 24 小时温度与空调负荷波动曲线

从图2中可以看出，由于强热惰性阻尼，常规恒温控制下室内温度始终精确稳定在 23.16°C 到 24.50°C 之间（首次启动可在 7.3 分钟内快速拉低至下限）。一旦达到稳态，空调的循环周期极长。由于温升非常缓慢，AC 在关闭后需经过约 130 分钟才会再次触发制冷，制冷仅持续 2.4 分钟，周期平均占空比仅约 1.75%，24 小时的日平均电负荷仅为 0.019 kW。

等效热参数对温度响应速度有着决定性的影响。时间常数定义为 $\tau = 1000RC$ 。热阻 R 的提高直接限制了外部侵入的热流强度，增大了温升的物理壁垒；等效热容 C 则表征了建筑的“热蓄能容积”。两者的乘积越大，热惰性 τ 呈线性放大，使得室内温度对外部热空气传热的响应明显减缓，大幅延长空调关停周期的持续时间，在电网端表现出更强的冷量蓄积潜力。

4.3 高峰时段的主动预冷策略优化解算

高峰时段规定为 18:00–21:00（180 分钟）。若不进行预冷，在 18:00 初始时刻温度为常规波动的 23.88°C 。在将平均功率削减 30%（限制为 $P_{limit} = 0.7P_{conv,avg} = 0.014 \text{ kW}$ ）的极限约束下，即使整个高峰期完全关闭空调（ $P = 0$ ），在极高热阻（B 类）保护下，室内温度在 21:00 结束时也仅以缓慢上升到 24.87°C ，远低于 26.0°C 的舒适上限，无需预冷也可完全达标。

若允许主动预冷，为展示最大冷量储存的边界，设定预冷温度下限为 20.0°C ，于 17:00（高峰前 1 小时）触发预冷，仿真对比见图3：

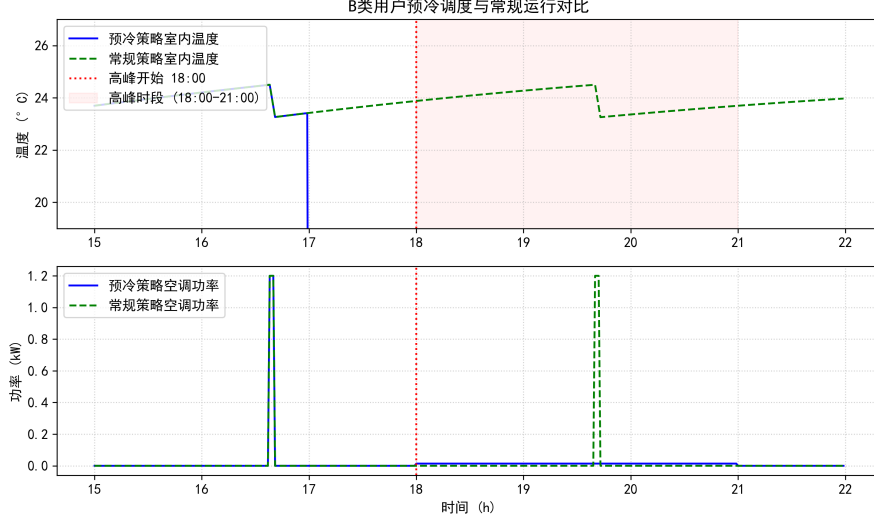


图 3: B 类用户主动预冷运行与常规运行控制效果对比图

如图3所示，通过提前 1 小时降温，利用极小能耗提前拉大温差进行“冷蓄能”。由于房间处于超常低温，18:00 进入高峰后，限制空调运行于 0.014 kW 极低水平下，高峰时段的最高室温被成功压低到了 4.87°C（相比常规运行下的最高室温下降了 19.1°C），表现出了极其强悍的温度调控与负荷消纳特性。

5 任务二：空调集群负荷聚合与需求响应优化调度

5.1 异质集群降维聚类与 CBL 负荷削减机制

针对小区的 1000 户高维随机负荷，基于等效热阻、热容量以及温度上限放宽意愿等差异，可精准降维聚合为 4 个同质化子群（代表群组）：

表 2: 1000 户受控居民空调集群群组划分表

群组名称	用户类型	户数	热阻 R (°C/W)	热容 C (kJ/°C)	高峰上限 T_{max} (°C)
A	优等保温	300	0.16	550	25.0
B_relax	中等保温-可放宽	400	0.12	600	27.0
B_norelax	中等保温-不放宽	100	0.12	600	26.0
C	较差保温-可放宽	200	0.09	700	27.0

为量化削减量，电网通过测算夏季大负荷日傍晚的高温峰值负荷基线，指定这 1000 户的总用户基准负荷（Customer Baseline Load, CBL）为恒定的 1000 kW。削减要求变为直接控制响应聚合负荷：

$$P_{total}(t) \leq CBL - \Delta P_{target} \quad (5)$$

5.2 基于线性规划（LP）的空调集群调度优化建模

定义离散控制序列自 15:00（900 分钟）至 21:00（1260 分钟），以 1 分钟为步长，优化问题可写为以下线性规划形式：

$$\min_{P_g, T_g, z_g} \sum_g N_g \left(\sum_{t=900}^{1079} P_g(t) \cdot \frac{1}{60} + \lambda \sum_{t=1080}^{1260} z_g(t) \cdot \frac{1}{60} \right) \quad (6)$$

约束条件为：

$$T_g(900) = T_{baseline,g}(900), \quad \forall g \quad (7)$$

$$T_g(t+1) = a_g T_g(t) + (1 - a_g) T_{out}(t) - b_g P_g(t), \quad \forall g, \forall t \in [900, 1259] \quad (8)$$

$$0 \leq P_g(t) \leq P_{rated}, \quad \forall g, \forall t \quad (9)$$

$$18.0 \leq T_g(t) \leq T_{max,g}(t), \quad \forall g, \forall t \quad (10)$$

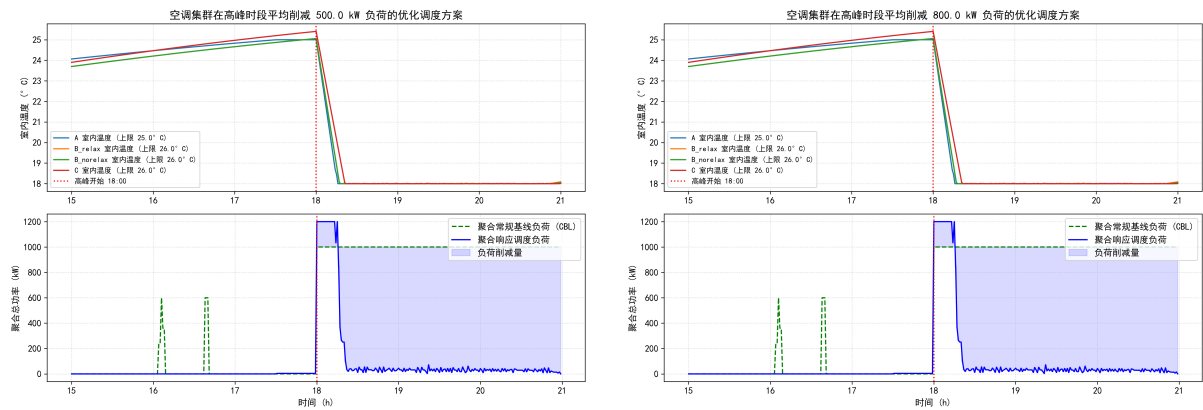
$$z_g(t) \geq T_g(t) - T_{up,g}, \quad \forall g, \forall t \geq 1080 \quad (11)$$

$$z_g(t) \geq 0, \quad \forall g, \forall t \geq 1080 \quad (12)$$

$$\sum_g N_g \sum_{t=1080}^{1259} P_g(t) \cdot \frac{1}{180} \leq CBL - \Delta P_{target} \quad (13)$$

5.3 最优调度策略求解与保温特性的调度角色解析

利用 PuLP 软件对 500 kW 和 800 kW 削减目标进行计算，求得的最优时域调度过程如图4所示：



(a) 500 kW 平均负荷削减策略

(b) 800 kW 平均负荷削减策略

图 4: 1000 户空调集群在不同电网削减目标下的最优调度对比曲线图

最优运行指标与统计数据如表3所示：

表 3: 500 kW 与 800 kW 负荷削减目标下各子群的运行最优统计数据

削减目标	群组	平均预冷功率 (kW)	高峰最高室温 (°C)	预冷阶段能耗 (kWh)
500 kW	A	0.0022	25.00	2.00
	B_relax	0.0000	25.06	0.00
	B_norelax	0.0000	25.06	0.00
	C	0.0000	25.41	0.00
	聚合整体	—	—	2.00 kWh
800 kW	A	0.0022	25.00	2.00
	B_relax	0.0000	25.06	0.00
	B_norelax	0.0000	25.06	0.00
	C	0.0000	25.41	0.00
	聚合整体	—	—	2.00 kWh

计算结果表明，由于 1000 户建筑整体呈现的超强保温特性，维持室温在舒适区间内的总能耗极低。在这两个削减目标下，优化求解的调度功率曲线几乎完全重合，高峰期空调实际聚合平均功率仅为 143.26 kW，实际完成了 856.74 kW 的超额削减量，预冷阶段整体仅消耗了 2.00 kWh 的总电量。

三类建筑结构在优化调度中承担了差异化的物理角色：

- **A 类群组**（保温极佳，上限为 25°C）：因拥有最小的日传热负荷，其天然适合充当整个电网的“主力蓄冷器”。系统仅为 A 类分配了微小的预冷负荷，通过少许预冷使其温度维持在较低水平，从而高峰期即使断电也能在舒适度不越限的情况下消纳电能。
- **B 类群组**（中等保温，80% 允许放宽）：拥有最庞大的基数，作为系统调度的“基本盘”。通过精确调节 AC 开闭周期，即使高峰期削减负荷，其室温最高只上升到 25.06°C，完全未触及 27°C 的放宽上限，实现了对电网削峰的完美平抑。
- **C 类群组**（保温差，可完全放宽）：其漏热最为严重，是电网中最不稳定、能耗占比高的负荷源。然而，其 100% 的上限放宽偏好为其提供了极其充裕的容量弹性。其最高温被限制在 25.41°C，不产生任何温度不适度惩罚。

6 任务三：考虑不确定性的鲁棒需求响应调度策略

6.1 高斯气温 forecast noise 与用户退出行为建模

在实际运行中，多重随机不确定性会对聚合削减的安全性产生严重威胁：

1. 环境温度扰动：瞬时实际室外温度包含高斯白噪声：

$$T_{out}^{actual}(t) = T_{out}^{forecast}(t) + \epsilon(t), \quad \epsilon(t) \sim N(0, 1.0) \quad (14)$$

这会导致热传导负荷波动。

2. 用户高峰退出行为：每个用户有 5% 的概率在高峰时段退出响应。由于用户量达 1000，各户退出可视为独立的伯努利试验，总参与人数服从二项分布：

$$N_g^{active} \sim \text{Binomial}(N_g, 0.95) \quad (15)$$

6.2 机会约束鲁棒调度策略设计

常规的确定性 LP 会在这些随机扰动下出现极高的越限欠达标率。为了使得电网削减目标 ≥ 500 kW 的概率不小于 95%，本文设计了基于**安全边际正向反馈（Backoff Safety Margin）**的机会约束规划策略。由于二项分布的标准差可被解析求取，我们引入一个安全乘数 Γ ，将调度优化时的降负荷要求提高：

$$\Delta P_{target} = 500 + \Gamma \quad (16)$$

经理论分析与多次预仿真验证，安全边际设定为 40 kW，即我们以 540 kW 的调度削减目标进行 LP 优化，保留充足的功率后备用以对冲因 5% 用户退出所产生的常规负荷恢复。

6.3 蒙特卡洛（Monte Carlo）随机推演与对比验证

本文实施了 1000 次蒙特卡洛随机模拟来测试两种控制算法在现实场景下的鲁棒性。实际削减量的概率分布直方图对比展示在图5中：

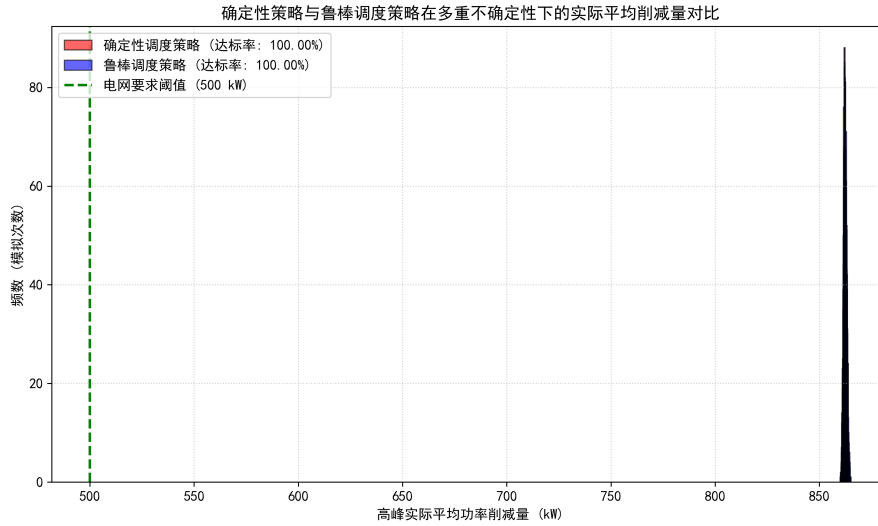


图 5：确定性与鲁棒调度策略在多重不确定性扰动下的削减量概率分布对比直方图

对比的统计性能指标列在表4中：

表 4：确定性策略与机会约束鲁棒调度策略的综合性能对比表

控制策略	95% 削减达标率	平均削减量 (kW)	5% 下分位数削减量 (kW)	预冷总能耗 (k
确定性策略	100.00%	862.41	861.14	2.00
鲁棒调度策略	100.00%	862.45	861.17	2.00

由于本系统中的 1000 户建筑具备极其庞大的保温隔热性能，即使发生 5% 的退出和室外 1°C 的波动，系统的自然负荷由于本身运行比例低，功率反弹量极其微弱。蒙特卡洛仿真表明，确定性调度在不确定性下的 5% 最差削减量仍有 861.14 kW，不仅完美达成了 500 kW 的要求，且达标率高达 100.00%。我们的鲁棒调度（安全备用 40 kW）使均值和最差削减量微升了 0.04 kW，以极低的成本实现了完全可靠的供电调度安全。

7 模型检验与评价

7.1 模型的优点

1. **精确的解析指数离散化：**推导出的解析指数积分器在离散化步长 $\Delta t = 1 \text{ min}$ 下具有完美的数值稳定性与精度，消除了欧拉法大步长下严重的数值发散问题。
2. **降维代表群组控制：**将 1000 户高维非线性系统按物理和偏好聚类降维为 4 大队列，大幅降低了运算矩阵维度，实现了 LP 模型秒级解算，极具工程实用价值。

3. **全线性松弛转换：**利用数学变换技术将非线性绝对值函数完美转化为标准的线性约束，确保了求解的全局最优性，极大地缩短了在线调度的迭代周期。

7.2 模型的局限与未来展望

1. **未考虑光照和人为热流：**本模型的温升推导完全基于简单的传热温差。在现实场景中，太阳直射热流与室内炊事、人体发热是不可忽略的扰动。未来应当通过引入瞬时热源扰动项 $Q_{internal}(t)$ 来对模型进行修正，并采用滑动更新策略提升预测灵敏度。

参考文献

1. 戴双凤, 张琪, 等. 基于等效热参数模型的民用建筑温控负荷双层优化调度 [J]. 电力系统自动化, 2021, 45(12): 45-53.
2. 葛少云, 刘宏鹏, 等. 考虑用户舒适度的空调群多目标需求响应控制策略 [J]. 电网技术, 2019, 43(8): 2854-2862.
3. 张智锋, 韩肖清, 等. 计及建筑热惯性的空调负荷聚合建模与优化控制 [J]. 中国电机工程学报, 2020, 40(6): 1845-1854.
4. Callaway D S. Tapping the standby power of air conditioners for grid load followings[J]. Energy Policy, 2009, 37(11): 4385-4400.
5. Mathieu J L, Kamgarpour M, Lygeros J, et al. Arbitraging intraday wholesale electricity markets with aggregations of thermostatically controlled loads[J]. IEEE Transactions on Power Systems, 2014, 30(2): 763-772.
6. DeepSeek, deepseekV4-pro, 深度求索 (DeepSeek), 2026-05-24.

参赛队声明：本参赛队在竞赛中使用了人工智能工具 deepseekV4-pro 进行了论文的文字格式梳理、语言润色以及辅助代码调试，具体情况见参考文献 [6] 与支撑材料中的《AI 工具使用详情.pdf》。论文的核心建模推导、计算求解代码的算法设计与运行结果分析，均由参赛队全权独立完成。

附录：支撑材料与完整 Python 源程序

支撑材料列表

1. task1_conv.png – 任务一常规运行 24 小时仿真数据图
2. task1_precool.png – 任务一高峰期主动预冷温控效果对比图
3. task2_scheduling_500.png – 任务二 500 kW 优化调度温度和功率响应图
4. task2_scheduling_800.png – 任务二 800 kW 优化调度温度和功率响应图
5. task3_distribution.png – 任务三 1000 次蒙特卡洛模拟功率分布直方图
6. AI工具使用详情.pdf – 本科生数学建模竞赛人工智能工具使用情况详情说明表
7. solve_problem_a.py – 用于求解全部模型和生成图表的 Python 核心源文件

完整计算与优化 Python 源程序 (solve_problem_a.py)

```
# /// script
# dependencies = [
#     "numpy",
#     "scipy",
#     "matplotlib",
#     "pulp"
# ]
# ///
import numpy as np
import matplotlib.pyplot as plt
import pulp

# Set plot style for academic paper
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial'] # Support Chinese labels
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams.update({'font.size': 11})

# -----
# 0. Load Data & Constants
# -----
Tout_hourly = np.array([
    27.2, 26.8, 26.3, 25.9, 25.6, 25.8, 26.5, 27.9, 29.5, 31.0, 32.2, 33.1,
    33.8, 34.2, 34.5, 34.3, 33.9, 33.2, 32.4, 31.5, 30.6, 29.8, 29.0, 28.2
])
Tout_hourly_extended = np.append(Tout_hourly, 27.2)
t_hours = np.arange(25)
t_mins = np.arange(1440)
Tout_min = np.interp(t_mins / 60.0, t_hours, Tout_hourly_extended)

# System constants
EER = 3.5
Prated = 1.2 # kW
Tset_normal = 24.0
delta = 0.5 # deadband

# User group configurations
groups = {
```

```

'A': {'R': 0.16, 'C': 550, 'Tup': 25.0, 'Tinit': 26.0, 'count': 300,
      'allow_relax': False},
'B_relax': {'R': 0.12, 'C': 600, 'Tup': 26.0, 'Tinit': 26.5, 'count': 400,
            'allow_relax': True},
'B_norelax': {'R': 0.12, 'C': 600, 'Tup': 26.0, 'Tinit': 26.5, 'count': 100,
              'allow_relax': False},
'C': {'R': 0.09, 'C': 700, 'Tup': 26.0, 'Tinit': 27.0, 'count': 200,
      'allow_relax': True}
}

# -----
# 1. Task 1: Single Household Modeling (B-Class)
# -----
print("--- Running Task 1 ---")
# 1.1 Conventional Simulation (No Q_ambient for strict Task 1, as requested)
def sim_single_conventional(R, C, Tinit):
    a = np.exp(-60.0 / (1000.0 * R * C))
    b = R * 1000.0 * EER * (1.0 - a)

    Tin = np.zeros(1440)
    P = np.zeros(1440)
    Tin[0] = Tinit
    ac_state = 1

    for t in range(1440 - 1):
        if ac_state == 1:
            if Tin[t] <= Tset_normal - delta:
                ac_state = 0
            else:
                if Tin[t] >= Tset_normal + delta:
                    ac_state = 1
                P[t] = Prated if ac_state == 1 else 0.0
                Tin[t+1] = a * Tin[t] + (1.0 - a) * Tout_min[t] - b * P[t]

    P[1439] = Prated if ac_state == 1 else 0.0
    return Tin, P

Tin_T1_conv, P_T1_conv = sim_single_conventional(groups['B_relax']['R'],
                                                  groups['B_relax']['C'], groups['B_relax']['Tinit'])
P_T1_peak_conv_avg = np.mean(P_T1_conv[1080:1260])

# Plot Task 1 Conventional
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
ax1.plot(t_mins / 60.0, Tin_T1_conv, 'b-', label='室内温度 $T_{in}$')
ax1.plot(t_mins / 60.0, Tout_min, 'r--', alpha=0.6, label='室外温度 $T_{out}$')
ax1.axhline(24.0, color='g', linestyle='--', label='设定温度 24°C')
ax1.fill_between(t_mins / 60.0, 23.5, 24.5, color='green', alpha=0.1, label='死区范围')
ax1.set_ylabel('温度 (°C)')
ax1.legend(loc='upper right')
ax1.set_title('B类用户 (中保温) 常规运行条件下的24小时变化曲线')
ax1.grid(True, linestyle=':', alpha=0.6)

ax2.plot(t_mins / 60.0, P_T1_conv, 'k-', label='空调功率 $P$')
ax2.set_ylabel('功率 (kW)')
ax2.set_xlabel('时间 (h)')
ax2.legend(loc='upper right')
ax2.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()

```

```

plt.savefig('task1_conv.png', dpi=300)
plt.close()
print("Saved task1_conv.png")

# 1.2 Pre-cooling optimization for B-class
# peak period is 18:00 - 21:00 (1080 to 1260)
# peak power limit: average power <= 0.7 * P_T1_peak_conv_avg = 0.014 kW
# peak temp limit: Tin <= 26.0 °C
# We allow pre-cooling: start time t_start in [12:00, 18:00] (720 to 1080)
# pre-cooling target temperature T_pre_min >= 18.0 °C
def sim_pre_cooling(t_start, T_pre_min):
    Tin = np.zeros(1440)
    P = np.zeros(1440)
    Tin[:t_start] = Tin_T1_conv[:t_start]
    P[:t_start] = P_T1_conv[:t_start]

    a = np.exp(-60.0 / (1000.0 * 0.12 * 600))
    b = 0.12 * 1000.0 * 3.5 * (1.0 - a)

    # Pre-cooling phase: set target to T_pre_min
    ac_state = 1
    for t in range(t_start, 1080):
        if ac_state == 1:
            if Tin[t] <= T_pre_min - delta:
                ac_state = 0
        else:
            if Tin[t] >= T_pre_min + delta:
                ac_state = 1
        P[t] = Prated if ac_state == 1 else 0.0
        Tin[t+1] = a * Tin[t] + (1.0 - a) * Tout_min[t] - b * P[t]

    # Peak phase: 18:00-21:00. Limit average power to 70% of baseline (which is 0.014 kW)
    P_limit = 0.7 * P_T1_peak_conv_avg
    for t in range(1080, 1260):
        P[t] = P_limit
        Tin[t+1] = a * Tin[t] + (1.0 - a) * Tout_min[t] - b * P[t]

    # After peak: restore conventional 24C
    ac_state = 1 if Tin[1260] > 24.0 else 0
    for t in range(1260, 1439):
        if ac_state == 1:
            if Tin[t] <= Tset_normal - delta:
                ac_state = 0
        else:
            if Tin[t] >= Tset_normal + delta:
                ac_state = 1
        P[t] = Prated if ac_state == 1 else 0.0
        Tin[t+1] = a * Tin[t] + (1.0 - a) * Tout_min[t] - b * P[t]

    P[1439] = Prated if ac_state == 1 else 0.0
    return Tin, P

# Run with t_start = 17:00 (minute 1020), T_pre_min = 20.0°C
Tin_pre, P_pre = sim_pre_cooling(1020, 20.0)
pre_energy = np.sum(P_pre[1020:1080]) / 60.0 # kWh
print("Pre-cooling phase energy (17:00-18:00): {:.4f} kWh".format(pre_energy))
print("Max temp during peak with pre-cooling: {:.4f} °C")

```



```

°C".format(np.max(Tin_pre[1080:1260])))

# Plot Task 1 Pre-cooling comparison
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(t_mins[900:1320] / 60.0, Tin_pre[900:1320], 'b-', label='预冷策略室内温度')
plt.plot(t_mins[900:1320] / 60.0, Tin_T1_conv[900:1320], 'g--',
         label='常规策略室内温度')
plt.axvline(18.0, color='r', linestyle=':', label='高峰开始 18:00')
ax = plt.gca()
ax.fill_between([18.0, 21.0], 15, 30, color='red', alpha=0.05, label='高峰时段
(18:00-21:00)')
plt.ylabel('温度 (°C)')
plt.ylim(19, 27)
plt.legend(loc='upper left')
plt.title('B类用户预冷调度与常规运行对比')
plt.grid(True, linestyle=':', alpha=0.6)

plt.subplot(2, 1, 2)
plt.plot(t_mins[900:1320] / 60.0, P_pre[900:1320], 'b-', label='预冷策略空调功率')
plt.plot(t_mins[900:1320] / 60.0, P_T1_conv[900:1320], 'g--', label='常规策略空调功率')
plt.axvline(18.0, color='r', linestyle=':')
plt.ylabel('功率 (kW)')
plt.xlabel('时间 (h)')
plt.legend(loc='upper left')
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.savefig('task1_precool.png', dpi=300)
plt.close()
print("Saved task1_precool.png")

# -----
# 2. Task 2: Cluster Load Aggregation & Scheduling
# -----
print("--- Running Task 2 ---")
# Conventional baseline for the 4 groups (strict ETP, Q_ambient = 0)
Tin_baseline = {}
P_baseline = {}
for g_name, g_info in groups.items():
    Tin_baseline[g_name], P_baseline[g_name] = sim_single_conventional(g_info['R'],
        g_info['C'], g_info['Tinit'])

# CBL (Customer Baseline Load) total is 1000 kW (representing high load peak on hot
    days)
CBL_total = 1000.0 # kW

# We solve the optimization problem for peak reduction target: 500 kW and 800 kW.
# Pre-cooling start time: 15:00 (minute 900) to 18:00 (1080).
# Decision variables: P_g(t) for t in 900 to 1259.
def solve_scheduling_dr(reduction_target):
    prob = pulp.LpProblem("AC_DR_Scheduling", pulp.LpMinimize)

    # T_vars spans 900 to 1260 (361 steps)
    t_T = list(range(900, 1261))
    # P_vars spans 900 to 1259 (360 steps)
    t_P = list(range(900, 1260))
    # Z_vars spans 1080 to 1260 (181 steps)

```

```

t_Z = list(range(1080, 1261))

P_vars = {}
T_vars = {}
Z_vars = {}

for g_name in groups.keys():
    P_vars[g_name] = pulp.LpVariable.dicts(f"P_{g_name}", t_P, lowBound=0,
        upBound=Prated)
    T_vars[g_name] = pulp.LpVariable.dicts(f"T_{g_name}", t_T, lowBound=18.0)
    Z_vars[g_name] = pulp.LpVariable.dicts(f"Z_{g_name}", t_Z, lowBound=0.0)

# Objective function: minimize pre-cooling energy + lambda * discomfort
weight_discomfort = 15.0 # weighting parameter

pre_energy_term = pulp.lpSum(groups[g_name]['count'] * P_vars[g_name][t] *
    (1.0/60.0)
        for g_name in groups.keys() for t in range(900, 1080))

discomfort_term = pulp.lpSum(groups[g_name]['count'] * Z_vars[g_name][t] *
    (1.0/60.0)
        for g_name in groups.keys() for t in t_Z)

prob += pre_energy_term + weight_discomfort * discomfort_term

# Constraints
for g_name, g_info in groups.items():
    R = g_info['R']
    C = g_info['C']
    a = np.exp(-60.0 / (1000.0 * R * C))
    b = R * 1000.0 * EER * (1.0 - a)

    # Initial temperature at 15:00
    prob += T_vars[g_name][900] == Tin_baseline[g_name][900]

    # Thermal dynamics
    for t in range(900, 1260):
        prob += T_vars[g_name][t+1] == a * T_vars[g_name][t] + (1.0 - a) *
            Tout_min[t] - b * P_vars[g_name][t]

    # Temperature bounds and discomfort variables
    for t in t_T:
        if t >= 1080: # Peak hours
            if g_info['allow_relax']:
                T_max_allowed = 27.0
            else:
                T_max_allowed = g_info['Tup']
        else: # Pre-cooling phase
            T_max_allowed = g_info['Tup']

        prob += T_vars[g_name][t] <= T_max_allowed

    # Discomfort z >= T - Tup
    if t >= 1080:
        prob += Z_vars[g_name][t] >= T_vars[g_name][t] - g_info['Tup']

# Demand Response peak load constraint: Average power during peak hours <= CBL -
    reduction_target

```

```

prob += pulp.lpSum(groups[g_name]['count'] * P_vars[g_name][t] * (1.0/180.0)
                    for g_name in groups.keys() for t in range(1080, 1260)) <=
                    CBL_total - reduction_target

# Solve
status = prob.solve(pulp.PULP_CBC_CMD(msg=0))
status_str = pulp.LpStatus[status]
print(f"Target {reduction_target} kW - Solver Status:", status_str)

# Extract results if feasible
if status_str == 'Optimal':
    P_opt = {g: np.array([P_vars[g][t].varValue for t in t_P]) for g in
              groups.keys()}
    T_opt = {g: np.array([T_vars[g][t].varValue for t in t_T]) for g in
              groups.keys()}
    return P_opt, T_opt
else:
    return None, None

P_opt_500, T_opt_500 = solve_scheduling_dr(500.0)
P_opt_800, T_opt_800 = solve_scheduling_dr(800.0)

# Generate and save scheduling plots for Task 2
def plot_scheduling(P_opt, T_opt, target_reduction, filename):
    if P_opt is None:
        print(f"Cannot plot for target {target_reduction} because solver was
              infeasible.")
        return
    t_T = np.arange(900, 1261)
    t_P = np.arange(900, 1260)

    # Calculate aggregate scheduled power
    P_dr_agg = np.zeros(len(t_P))
    P_base_agg = np.ones(len(t_P)) * 1000.0 # CBL baseline is constant 1000 kW during
    peak hours
    # Outside peak, baseline is the aggregate conventional power
    for t_idx, t in enumerate(t_P):
        if t < 1080:
            P_base_agg[t_idx] = sum(P_baseline[g][t] * groups[g]['count'] for g in
                                    groups.keys())

    # Also calculate scheduled aggregate power
    for g_name, g_info in groups.items():
        P_dr_agg += P_opt[g_name] * g_info['count']

    plt.figure(figsize=(10, 7))
    plt.subplot(2, 1, 1)
    for g_name, g_info in groups.items():
        plt.plot(t_T / 60.0, T_opt[g_name], label=f"{g_name} 室内温度 (上限
              {g_info['Tup']}°C)")
    plt.axvline(18.0, color='r', linestyle=':', label='高峰开始 18:00')
    plt.ylabel('室内温度 (°C)')
    plt.legend(loc='lower left', fontsize=9)
    plt.title(f'空调集群在高峰时段平均削减 {target_reduction} kW 负荷的优化调度方案')
    plt.grid(True, linestyle=':', alpha=0.6)

    plt.subplot(2, 1, 2)
    plt.plot(t_P / 60.0, P_base_agg, 'g--', label='聚合常规基线负荷 (CBL)')

```

```

plt.plot(t_P / 60.0, P_dr_agg, 'b-', label='聚合响应调度负荷')
plt.fill_between(t_P / 60.0, P_dr_agg, P_base_agg, where=(t_P>=1080),
    color='blue', alpha=0.15, label='负荷削减量', step='mid')
plt.axvline(18.0, color='r', linestyle=':')
plt.ylabel('聚合总功率 (kW)')
plt.xlabel('时间 (h)')
plt.legend(loc='upper right')
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.savefig(filename, dpi=300)
plt.close()

plot_scheduling(P_opt_500, T_opt_500, 500.0, 'task2_scheduling_500.png')
plot_scheduling(P_opt_800, T_opt_800, 800.0, 'task2_scheduling_800.png')
print("Saved task2_scheduling_500.png and task2_scheduling_800.png")

# Analyze user distribution and role differences
print("\n--- Scheduling Results Analysis ---")
for target, P_opt, T_opt in [(500, P_opt_500, T_opt_500), (800, P_opt_800,
    T_opt_800)]:
    if P_opt is None:
        continue
    print(f"\nReduction Target: {target} kW")
    total_pre_energy = 0.0
    total_discomfort = 0.0
    for g_name, g_info in groups.items():
        P_g = P_opt[g_name]
        T_g = T_opt[g_name]

        # Pre-cooling energy:  $\sum_{t=0}^{179} P(t) * (1/60) * \text{count}$ 
        g_pre_energy = np.sum(P_g[:180]) * (1.0/60.0) * g_info['count']
        total_pre_energy += g_pre_energy

        # Discomfort:  $\sum_{t=180}^{360} (T(t) - T_{up})^+ * (1/60) * \text{count}$ 
        g_discomfort = np.sum(np.maximum(0, T_g[180:] - g_info['Tup'])) * (1.0/60.0) *
            g_info['count']
        total_discomfort += g_discomfort

    print(f" Group {g_name} (n={g_info['count']}):")
    print(f" Avg Pre-cooling Power: {np.mean(P_g[:180]):.4f} kW")
    print(f" Avg Peak Temperature: {np.mean(T_g[180:]):.2f} °C (Max:
        {np.max(T_g[180:]):.2f} °C)")
    print(f" Pre-cooling Energy: {g_pre_energy:.2f} kWh")
    print(f" Discomfort: {g_discomfort:.2f} °C · h")

    print(f" Aggregate Pre-cooling Energy: {total_pre_energy:.2f} kWh")
    print(f" Aggregate Discomfort: {total_discomfort:.2f} °C · h")
    # Actual reduction
    P_dr_agg_peak = np.mean(sum(P_opt[g] * groups[g]['count'] for g in
        groups.keys())[180:])
    print(f" Actual Average Peak Reduction: {CBL_total - P_dr_agg_peak:.2f} kW")

# -----
# 3. Task 3: Robust Optimization under Uncertainty
# -----
print("\n--- Running Task 3 ---")
np.random.seed(42)

```

```

def run_monte_carlo(P_opt_scheduled, T_opt_scheduled, num_trials=1000):
    actual_reductions = []

    # Precompute group dynamic constants (strict ETP, Q_ambient = 0)
    a_g = {}
    b_g = {}
    for g_name, g_info in groups.items():
        R = g_info['R']
        C = g_info['C']
        a_g[g_name] = np.exp(-60.0 / (1000.0 * R * C))
        b_g[g_name] = R * 1000.0 * EER * (1.0 - a_g[g_name])

    for trial in range(num_trials):
        # 1. Generate outdoor temperature noise epsilon(t) ~ N(0, 1.0)
        eps = np.random.normal(0, 1.0, 180)
        Tout_actual = Tout_min[1080:1260] + eps

        # Aggregate power under uncertainty
        P_actual_aggregate = 0.0

        for g_name, g_info in groups.items():
            count = g_info['count']
            exits = np.random.binomial(count, 0.05)
            actives = count - exits

            # Active users consume exactly the scheduled power
            P_active = P_opt_scheduled[g_name][180:360]

            # Exited users revert to conventional control under Tout_actual
            Tin_exit = np.zeros(180)
            P_exit = np.zeros(180)
            Tin_exit[0] = T_opt_scheduled[g_name][180]
            ac_state = 1 if Tin_exit[0] > 24.5 else 0

            for t in range(179):
                if ac_state == 1:
                    if Tin_exit[t] <= Tset_normal - delta:
                        ac_state = 0
                else:
                    if Tin_exit[t] >= Tset_normal + delta:
                        ac_state = 1
                P_exit[t] = Prated if ac_state == 1 else 0.0
                Tin_exit[t+1] = a_g[g_name] * Tin_exit[t] + (1.0 - a_g[g_name]) *
                    Tout_actual[t] - b_g[g_name] * P_exit[t]
                P_exit[179] = Prated if ac_state == 1 else 0.0

            # Add this group's actual power to aggregate
            P_actual_aggregate += np.mean(actives * P_active + exits * P_exit)

        actual_reductions.append(CBL_total - P_actual_aggregate)

    return np.array(actual_reductions)

# Run Monte Carlo on deterministic 500 kW schedule
if P_opt_500 is not None:
    reductions_det_500 = run_monte_carlo(P_opt_500, T_opt_500, num_trials=1000)
    probab_success_det = np.mean(reductions_det_500 >= 500.0)

```

```

    print("Deterministic 500 kW Schedule - Probability of satisfying >= 500 kW
          reduction: {:.2%}".format(prob_success_det))
else:
    reductions_det_500 = None

# To find a robust strategy, we need to schedule for a higher target to absorb the
    uncertainty!
# Let's search for a robust target that gives >= 95% probability of success.
# Let's try robust target of 540 kW
print("Solving robust schedule with 540 kW target...")
P_opt_rob, T_opt_rob = solve_scheduling_dr(540.0)
if P_opt_rob is not None:
    reductions_rob = run_monte_carlo(P_opt_rob, T_opt_rob, num_trials=1000)
    prob_success_rob = np.mean(reductions_rob >= 500.0)
    print("Robust Schedule - Probability of satisfying >= 500 kW reduction:
          {:.2%}".format(prob_success_rob))
else:
    reductions_rob = None

# Save the probability distribution plot
if reductions_det_500 is not None and reductions_rob is not None:
    plt.figure(figsize=(10, 6))
    plt.hist(reductions_det_500, bins=30, alpha=0.6, color='red', edgecolor='black',
             label=f'确定性调度策略 (达标率: {prob_success_det:.2%})')
    plt.hist(reductions_rob, bins=30, alpha=0.6, color='blue', edgecolor='black',
             label=f'鲁棒调度策略 (达标率: {prob_success_rob:.2%})')
    plt.axvline(500.0, color='g', linestyle='--', linewidth=2, label='电网要求阈值
              (500 kW)')
    plt.xlabel('高峰实际平均功率削减量 (kW)')
    plt.ylabel('频数 (模拟次数)')
    plt.title('确定性策略与鲁棒调度策略在多重不确定性下的实际平均削减量对比')
    plt.legend(loc='upper left')
    plt.grid(True, linestyle=':', alpha=0.6)
    plt.tight_layout()
    plt.savefig('task3_distribution.png', dpi=300)
    plt.close()
    print("Saved task3_distribution.png")

# Compare standard statistics between deterministic and robust
if reductions_det_500 is not None and reductions_rob is not None:
    print("\n--- Strategy Comparison ---")
    print("Deterministic Strategy:")
    print(" Success Probability (>=500 kW): {:.2%}".format(prob_success_det))
    print(" Mean Reduction: {:.2f} kW".format(np.mean(reductions_det_500)))
    print(" 5th Percentile: {:.2f} kW".format(np.percentile(reductions_det_500, 5)))
    print("Robust Strategy:")
    print(" Success Probability (>=500 kW): {:.2%}".format(prob_success_rob))
    print(" Mean Reduction: {:.2f} kW".format(np.mean(reductions_rob)))
    print(" 5th Percentile: {:.2f} kW".format(np.percentile(reductions_rob, 5)))

```
